

class06

Nicolas Song

Q1a: Write a function `add()` to add some numbers together. Make sure to add comment(s) to the function explaining why you do things. Your first version of the function should add two input numbers together; e.g. `add(4, 7)` should return 11.

```
add_2 <- function(x, y) {  
  # Adding variables x and y  
  x + y  
}
```

Testing my function:

```
add_2(4, 7)
```

```
[1] 11
```

Q1b. Your second version of the function should add any numbers together that the user inputs; e.g. `add(1, 2, 3, -4)` should return 2.

```
add_multi <- function(x, ...) {  
  # Using sum because adding multiple numbers  
  sum(x, ...)  
}
```

```
add_multi(1,2,3,-4)
```

```
[1] 2
```

Q2: Write a function `generate_dna()` that generates a random DNA sequence of a length input by the user; e.g. `generate_dna(11)` may return “ATTTAAGGCTG”. Make sure to add comments to the function. Play with functions `sample()` and `paste()` in your R-Studio Console to see how they can help here.

```
generate_dna <- function(x) {
  # generating a nucleotide vector
  nucs <- c("A", "T", "G", "C")
  # using sample to pull out random nucleotides
  nucs_vector <- sample(nucs, x, replace=TRUE)
  # Generating a dna string from nucs_vector
  paste(nucs_vector, collapse="")
}

# testing my generate_dna() function
generate_dna(10)
```

```
[1] "TAACCGTCCC"
```

Q3: Write a function `generate_protein()` that generates a random peptide/protein sequence of a length input by the user; e.g. `generate_protein(6)` may return “WQRTAG”. Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function.

```
generate_protein <- function(n) { # generating a protein vector
  amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K",
    "M", "F", "P", "S", "T", "W", "Y")
  # using sample to pull out random amino acids
  amino_acids_vector <- sample(amino_acids, n, replace=TRUE)
  # generating a protein from amino_acids_vector
  paste(amino_acids_vector, collapse="")
}

# testing generate_protein() function
generate_protein(6)
```

```
[1] "AHPAEP"
```

Q4: Use your `generate_protein()` function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function `apply()` for this.

```
# creating a vector for the desired lengths
lengths <- c(5, 7, 9, 11)
# using sapply() to generate peptides
sapply(lengths, generate_protein)
```

```
[1] "ARTCC"      "LQCISCR"    "GGRWFATQQ"  "CPLSLEMDHTI"
```

Q5: Take each of your peptides from Q4 and run a BLASTP search (<https://blast.ncbi.nlm.nih.gov/Blast.cgi>) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, 100% Query Cover times 85.71% Per. Ident = $100\% \times 85.71\% = 1.00 \times 0.8571 \times 100\% = 85.71\%$); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment). Also, for each datapoint, add your initials in the “Your__Initials” column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph).

```
# creating a function for query cover times percent identity
max_ident_perc <- function(q, p) {
  # multiplying query cover and percent identity
  q*(p/100)
}

# testing for "LQREK"
max_ident_perc(100, 100.00)
```

```
[1] 100
```

```
# testing for "YESNLHL"
max_ident_perc(86, 83.33)
```

```
[1] 71.6638
```

```
# testing for "FEERLTDAF"  
max_ident_perc(89, 75.00)
```

```
[1] 66.75
```

```
#testing for "SCKCPANRPTA"  
max_ident_perc(100, 72.73)
```

```
[1] 72.73
```

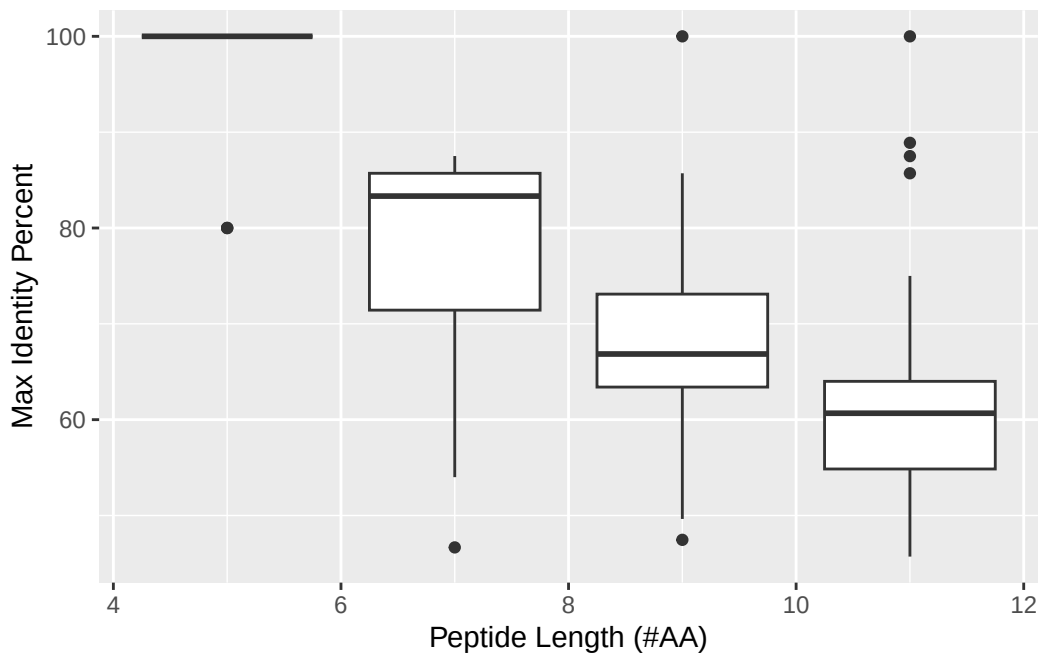
Q6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command `peptide_df <- read.csv("your_filename")`. Use the `head()` function to list the first 10 rows (note that the default for `head()` is 6) of the `peptide_df` dataframe as a test that everything looks good.

```
# uploading csv file into data frame  
peptide_df <- read.csv("BIMM143_peptide.csv")  
  
# isolating only the three desired columns  
peptide_df <- peptide_df[,1:3]  
  
# list first 10 rows  
head(peptide_df, n=10)
```

	Length_aa	Max_Identity_perc	Your_Initials
1	5	100.00	JLA
2	7	85.71	JLA
3	9	66.85	JLA
4	11	55.00	JLA
5	5	80.00	SKB
6	7	85.71	SKB
7	9	66.85	SKB
8	11	64.00	SKB
9	5	100.00	KET
10	7	85.71	KET

Q7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as `as.factor(Length_aa)`. Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the `labs()` function and/or using your favorite theme using a `theme()` function (see the `data-visualization_ggplot.pdf` cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (`p <- ggplot(peptide_df) +....etc`).

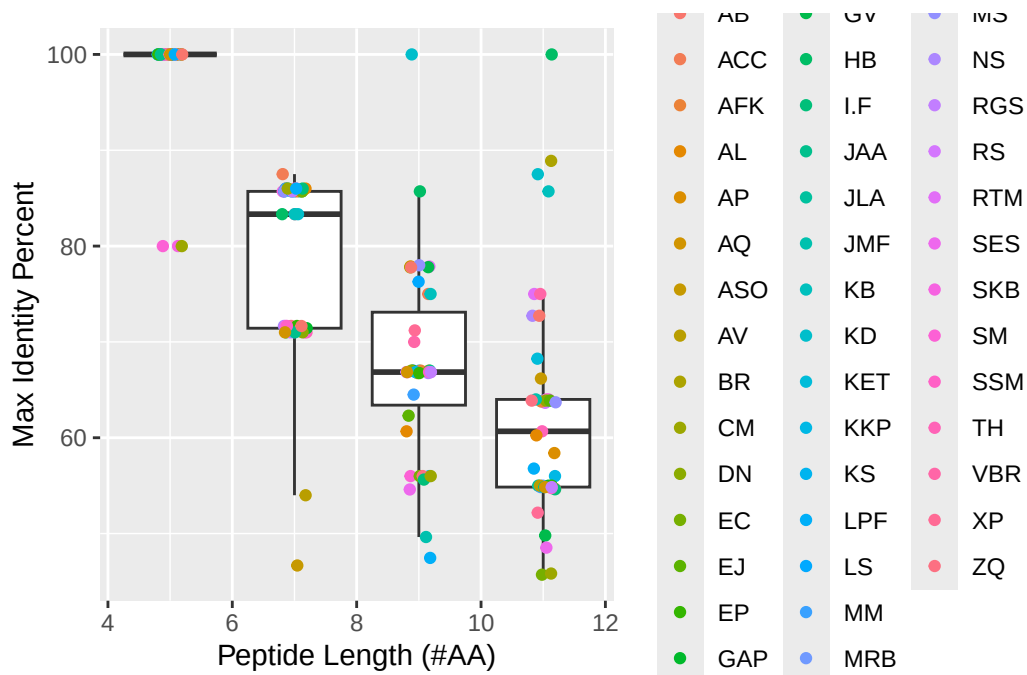
```
library(ggplot2)
# creating boxplot
ggplot(peptide_df, aes(x=Length_aa, y=Max_Identity_perc,
# grouping points
group=Length_aa)) + geom_boxplot() +
# labeling axes
labs(x="Peptide Length (#AA)", y="Max Identity Percent")
```



Q8a: Let's add a `geom_jitter()` layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points. First, add a `geom_jitter()` layer to your box plot with each data point colored according to students' initials by adding the following

additional line to your plot: `+ geom_jitter(aes(col=Your_Initials))`. Feel free to narrow the spread of the jitter points using the width argument (e.g. `width = 0.2`). You can find your own datapoints this way via the color legend, but it looks rather messy.

```
ggplot(peptide_df, aes(x=Length_aa, y=Max_Identity_perc, group=Length_aa)) +
  # removing outliers from geom_boxplot()
  geom_boxplot(outlier.shape = NA) +
  labs(x="Peptide Length (#AA)", y="Max Identity Percent") +
  # adding geom_jitter()
  geom_jitter(width=0.2, aes(col=Your_Initials))
```



Q8bi: Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the `geom_jitter()` call. First, use the `rep()` function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the `peptide_df` data frame. Assign this vector to an object you name `color`: i.e. `color <- c(rep(,))` (fill in the blanks). Once you have generated the color object, use the `table()` function to ensure that you have the correct number of instances of "GREY" in `color`. [1 pt] Hints for (i): Use your R-Studio Console to test the outputs of `rep("GREY",`

2), `dim(peptide_df)`, and `dim(peptide_df)[1]`, and make sure to look at documentation for the `dim()` function (`?dim`) to learn what the output numbers represent.

```
color <- c(rep("GREY",
# how many datapoints are there?
dim(peptide_df)[1]))

# ensuring the correct number of "GREY" instances
table(color)
```

```
color
GREY
172
```

Q8bii: Next, use the `Your_Initials` column in the `peptide_df` dataframe (which can be called as `peptide_df$Your_Initials`) to generate a `TRUE/FALSE` vector indicating which datapoints are yours (`TRUE`) and which are not (`FALSE`). Assign this vector to an object you name `mydata`: i.e. `mydata <- _____` (fill in the blank). Again, check that you have the correct number of `TRUE` and `FALSE` in `mydata` using the `table()` function. [1 pt] Hint for (ii): Use your R-Studio Console to test the outputs of `c("JLA", "MK", "BG") == "JLA"` and of `peptide_df$Your_Initials`

```
# generating my datapoints
mydata <- peptide_df$Your_Initials=="NS"

# testing how many datapoints are mine
table(mydata)
```

```
mydata
FALSE  TRUE
168     4
```

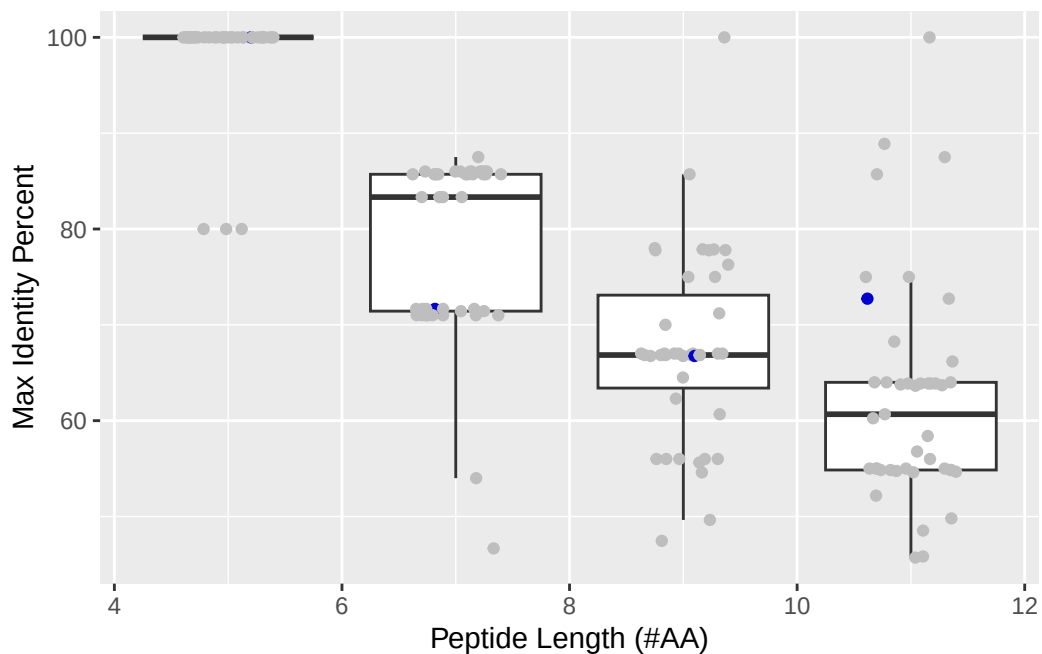
Q8biii: Now, use the `mydata` vector to replace the “GREY” values in the `color` vector that correspond to your data points with your favorite color, like this: `color[mydata] <- "COLOR"` (replace “COLOR” with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace “GREY” with your favorite color for all the positions that are `TRUE` in `mydata`, i.e. the positions of your data points. Again, check the vector with `table()`.

```
# coloring mydata
color[mydata] <- "mediumblue"
table(mydata)
```

```
mydata
FALSE TRUE
168    4
```

Q8biv: Now you are ready to remake the plot by adding the following line to your original box plot from Q7: `+ geom_jitter(col=color)`. (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. `width = 0.2`).

```
ggplot(peptide_df, aes(x=Length_aa, y=Max_Identity_perc, group=Length_aa)) + geom_boxplot(outlier.colour="red") +
  # highlighting my datapoints
  geom_jitter(col=color, width=0.4, aes(col=Your_Initials))
```



Q8c: Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s).

Yes, my peptide with a length of 11 amino acids

Q9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids.

Based on the findings in Q7 and 8, MHC class II molecules display 9 amino acid peptides because that is the optimal length for which they will target foreign cells rather than self. These peptides would not be 5 amino acids because at that length, they would target human (self) cells as shown by almost all of the 5 AA peptides hitting on 100% Max Identity Percent. 9 amino acids is thus optimal for MHC class II molecule selectivity.